26t1_NPPE1 Report

- `Name:` Harishwaran
- `roll no:` 22f1001159

Environment and Library setup:

```
import os
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
from sklearn.model_selection import train_test_split
import wandb
from tqdm import tqdm
import gc

# Configure W&B for experiment tracking
os.environ["WANDB_API_KEY"] = "wandb_v1_58Rfj8xGlV6vgWNYKAp7jtO7J3x_62JwVgzt2MFUIudaGBRKQ5ZexePTWh4neZ
run = wandb.init(
    entity="22f1001159-iit-madras",
    project="22f1001159_nppe1",
    name="resnet34_final_stable_run",
    config={"lr": 1e-4, "epochs": 30, "batch_size": 8, "img_size": 224}
)
config = run.config
```

Handling class imbalance

```
# Load data and extract class names
train_df = pd.read_csv("/kaggle/input/competitions/26-t-1-dl-gen-ainppe-1/train.csv")
```

```
class_names = train_df.columns[1:].tolist()

# Calculate inverse frequency weights with log-smoothing
counts = train_df[class_names].sum().values
weights = 1.0 / np.log1p(counts)
weights = weights / weights.sum() * 20  # Normalize across 20 classes
class_weights = torch.tensor(weights, dtype=torch.float).to('cuda')
```

Data Pipeline

```
class ChestDataset(Dataset):
    def __init__(self, df, img_dir, transform=None, is_test=False):
        self.df, self.img_dir, self.transform, self.is_test = df, img_dir, transform, is_test

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        name = self.df.iloc[idx, 0]
        img_path = os.path.join(self.img_dir, name if name.endswith('.png') else f"{name}.png")
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        if self.is_test:
            return image, torch.tensor(0), name # Dummy label for test set

        # Extract the index of the active pathology (One-Hot to Integer)
        label = torch.tensor(np.argmax(self.df.iloc[idx, 1:].values), dtype=torch.long)
        return image, label, name
```

Model Architecture and optimization

```
# Load Pretrained ResNet34
model = models.resnet34(weights='IMAGENET1K_V1')
model.fc = nn.Linear(model.fc.in_features, 20)
model = model.to('cuda')
```

```
# Weighted CrossEntropy with Label Smoothing (0.1) for better generalization
criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=config.lr)
```

Training loop with Gradient Accumulation

```
accumulation_steps = 4

for epoch in range(config.epochs):
    model.train()
    total_loss = 0
    optimizer.zero_grad()

    for i, (imgs, labels, _) in enumerate(tqdm(train_loader, desc=f"Epoch {epoch+1}")):
        imgs, labels = imgs.to('cuda'), labels.to('cuda')

        outputs = model(imgs)
        loss = criterion(outputs, labels) / accumulation_steps
        loss.backward()

        if (i + 1) % accumulation_steps == 0:
            optimizer.step()
            optimizer.zero_grad()

        total_loss += loss.item() * accumulation_steps

    # Memory management to prevent kernel restarts
    gc.collect()
    torch.cuda.empty_cache()

    avg_loss = total_loss / len(train_loader)
    wandb.log({"epoch": epoch+1, "loss": avg_loss})
```

Inference

```
model.eval()
results = []
test_df = pd.read_csv("/kaggle/input/competitions/26-t-1-dl-gen-ainppe-1/test.csv")
test_loader = DataLoader(ChestDataset(test_df, IMG_DIR, transform, is_test=True), batch_size=8)
```

```
with torch.no_grad():
    for imgs, _, ids in tqdm(test_loader):
        outputs = model(imgs.to('cuda'))
        preds = torch.argmax(outputs, dim=1).cpu().numpy()
        for i, p in enumerate(preds):
            row = [0]*20
            row[p] = 1 # Exactly one pathology predicted
            results.append([ids[i]] + row)

pd.DataFrame(results, columns=['id']+class_names).to_csv("submission.csv", index=False)
```